

Spring AI - Part 2

Multiple tools, system prompts, prompt templates, and the conversation id explained

WHAT THIS SESSION IS

A shorter, half length class (there was an in person meetup the same day). It goes deeper on tool calling with several tools working together, introduces system prompts and prompt templates, and finally answers all the questions about the **conversation id** that were left open from Session 03.

CONTENTS

1. Who actually calls the tool
2. Tool names and the @Tool / @ToolParam annotations
3. Adding more tools (weather, news)
4. RestTemplate versus WebClient versus FeignClient
5. System prompt versus user prompt
6. Many tools working together, orchestrated by the model
7. Honest failures matter (no hallucination)
8. Prompt templates
9. The conversation id, fully explained
10. Controlling memory and token cost
11. Key takeaways

1 Who actually calls the tool

A point worth nailing down, straight from the official docs: **the LLM decides to call a tool, and Spring AI executes it.** Spring AI is the facilitator; the model is the decision maker.

- When you make a tool available, its **definition** (name and description) is included in the chat request sent to the model.
- When the model decides it needs a tool, it responds with the tool name and the input parameters.
- Spring AI then runs that tool, gets the result, and feeds it back to the model.

WHY THE MODEL CAN EVEN KNOW A TOOL EXISTS

The AI model is external to your application. It cannot peek into your code. So *you* must register the tools in the request. If you do not add them, the model has no idea they exist.

2 Tool names and the @Tool / @ToolParam annotations

The **method name** of a `@Tool` method becomes the tool's name. Plain Java happily allows two methods with the same name (overloading), but Spring AI does **not** allow two tools with the same name. Doing so throws an error, because the name (plus description) is what identifies the tool to the model.

Annotation	Describes
<code>@Tool(description = "...")</code>	What the whole tool does (for example "get the weather information")
<code>@ToolParam(description = "...")</code>	What a single parameter means (for example "name of the city")

Both the tool description and the parameter descriptions genuinely help the model decide when and how to call the tool.

3 Adding more tools (weather, news)

You can register as many tools as your app needs; there is no limit of one or two. In the class three tools were built: a date/time tool (with a time zone parameter), a news tool, and a weather tool. A tool is just a `@Component` bean method:

```

@Component
public class WeatherTool {

    private final RestTemplate restTemplate = new RestTemplate();
    private final String apiKey = "YOUR_WEATHER_API_KEY";

    @Tool(description = "Get the current weather for a city")
    public String getCurrentWeather(@ToolParam(description = "city name")
String location) {
        try {
            String url = "https://some-weather-api/current?q=" + location +
"&key=" + apiKey;
            return restTemplate.getForObject(url, String.class);
        } catch (Exception e) {
            return "Error in fetching the weather";
        }
    }
}

```

Then inject the tool bean and register it on the call, alongside the others:

```

@Autowired private DateTimeTool dateTimeTool;
@Autowired private NewsTool newsTool;
@Autowired private WeatherTool weatherTool;

String answer = chatClient.prompt(prompt)
    .tools(dateTimeTool, newsTool, weatherTool) // register all of them
    .call()
    .content();

```

REGISTERING THE BEAN REGISTERS ITS TOOLS

You inject the component bean, and every `@Tool` method inside that bean gets registered. Adding a fourth, fifth, or sixth tool is just more of the same.

4 RestTemplate versus WebClient versus FeignClient

A tool that calls an external API needs an HTTP client. This is ordinary Spring Boot, not specific to Spring AI. Quick guide:

Client	Use when
RestTemplate	Traditional, simple synchronous REST calls
WebClient	You need reactive, non blocking, synchronous or asynchronous calls (from WebFlux). Not needed in this course.
FeignClient	Microservice environments, where you call registered services by name rather than a raw URL

5 System prompt versus user prompt

User prompt	System prompt
What the end user wants ("tell me about Spring AI")	Sets the context, tone, and behaviour of the model ("you are a smart personal assistant with access to date, weather, and news tools")
Comes from the user	Written by you, the developer

BEST PRACTICE

Always add a system prompt, whether or not the user gives a prompt. It controls *how* the AI answers (the tone and role), while the user prompt is *what* to answer. "Tell me about Java" gives a generic answer; "Act as a senior Java engineer, then tell about Java" gives a much better one. You can and should mix both.

```
String answer = chatClient.prompt()
    .system("You are a smart personal assistant. You have access to tools
for "
    + "date/time, weather, and latest news. Use whichever tools you
need to "
    + "answer clearly and completely.")
    .user(userPrompt)
    .tools(dateTimeTool, weatherTool, newsTool)
    .call()
    .content();
```

6 Many tools working together, orchestrated by the model

With three tools registered and a good system prompt, a single request like *"I'm visiting Pune tomorrow, tell me the date, weather, and latest news"* makes the model call all three tools, one after another, then blend their results with its own trained knowledge (for example nearby hotels) into one coherent answer.

THE DEVELOPER PLANS NOTHING

You never wrote code saying "first call the date tool, then the weather tool". With one prompt, the model itself decides which tools to call and in what order. You are only the enabler. Also, if a prompt does not need any tool (like "tell me a joke"), the model calls none, even with many tools registered.

7 Honest failures matter (no hallucination)

In the demo the news tool failed (a third party API issue), and the model answered honestly: "I was not able to retrieve the latest news due to technical issues, you can check local news sites." It did **not** invent fake news.

WHY THIS IS CRITICAL

For enterprise grade applications, an honest "I could not fetch this" is far safer than confidently made up data. Tool calling helps the model stay grounded instead of hallucinating.

8 Prompt templates

Sometimes you do not want the user to type a full prompt at all, only a few keywords. A **prompt template** is a reusable prompt with placeholders that you fill in with those keywords. Think of a Netflix style recommendation, or a "generate my learning roadmap" form where the user just clicks a few options.

```

@GetMapping("/recommend/movie")
public String recommendMovie(@RequestParam String type,
                             @RequestParam String year,
                             @RequestParam String language) {

    PromptTemplate template = new PromptTemplate(
        "I want to watch a {type} movie tonight from around {year} "
        + "with good ratings, in {language}. Suggest one and say why.");

    Prompt prompt = template.create(
        Map.of("type", type, "year", year, "language", language));

    return chatClient.prompt(prompt).call().content();
}

```

THE KEY DISTINCTION (A COMMON POINT OF CONFUSION)

A **system prompt** only sets the tone/role and is optional. A **prompt template** builds the *whole* prompt on the backend from small user inputs. Here the user never types a sentence; they pass keywords like `type=comedy`, `year=2011`, `language=English`, and your app assembles the full prompt. The same template dynamically produces a different final prompt for every user.

9 The conversation id, fully explained

This section answers every question raised in Session 03 about the conversation id used with the memory advisor.

- **It is just a String.** It can be a UUID (as used in the demo), a username, a session id, or any unique value. The UUID in the last class was only a quick way to generate a value, nothing more; it has no special link to AI.
- **It is required in stable Spring AI 1.0** (version 1.0.8 in class). Older/pre stable versions auto generated a default (`ChatMemory.DEFAULT_CONVERSATION_ID`); that default was removed. Forgetting it now throws "conversation id cannot be null".
- **It does NOT go to the LLM.** The model never sees it. Spring AI uses it purely, internally, to look up the history for that conversation. The "remembering" is done by Spring AI, not by the model.

- **It is once per user (or per session), not per API call.** If you generated a new id on every call, memory would be useless because each call would start fresh. Reuse the same id for follow up messages in the same chat.
- **It must be unique per user.** A single static id shared by all users would mix everyone's conversations. In a real app the id comes from the authentication layer (username plus a session id, for example `heather-123` , to stay unique).

HOW THE MEMORY ADVISOR USES THE ID

When you send a message with a conversation id, `MessageChatMemoryAdvisor` looks up all stored messages for that id, prepends them to your new prompt, and sends the combined history to the model. Then it stores the model's reply back under the same id. That is the whole "memory" mechanism.

In memory versus persistent memory

The demo used an **in memory** store, so the whole chat lives in RAM and is wiped when the app stops. For production and for truly stateful agents, swap the in memory chat memory for a persistent repository: **Redis, MongoDB, Cassandra, or JDBC**. The conversation id is the key in all of them.

10 Controlling memory and token cost

Every request is billed **per token**, including the history you send. If you resent the entire conversation each time, the cost would blow up and you could hit the model's context window limit.

That is why the class used `MessageWindowChatMemory` , a **sliding window** that keeps only the last N messages instead of the full history. You can set the window size:

```
ChatMemory chatMemory = MessageWindowChatMemory.builder()
    .maxMessages(20)    // keep only the last 20 messages
    .build();
```

THE TRADE OFF

More history remembered means more tokens per call, which means higher cost and slower responses. Bigger memory and lower cost pull against each other, so choose a window that fits your use case. Also note: too many registered tools (think thousands) increases app startup time, since Spring creates all the tool beans as singletons at startup. Do not expose thousands of tools to one agent.

Checking token usage via metadata

```
ChatResponse response = chatClient.prompt(prompt).call().chatResponse();
long totalTokens = response.getMetadata().getUsage().getTotalTokens();
```

11 Key takeaways

- The **model decides** which tools to call; Spring AI only executes them. You just register the tools.
- A tool's method name is its name; names must be unique. `@Tool` describes the tool, `@ToolParam` describes a parameter.
- Register many tools; the model calls only the ones a given prompt needs, and orchestrates them itself.
- Always add a **system prompt** to control tone and role; use a **prompt template** to build full prompts from a few user keywords.
- Good tools fail honestly instead of hallucinating, which matters for enterprise apps.
- The **conversation id** is a unique string per user/session, required in stable Spring AI 1.0, never sent to the model, and used by Spring AI to look up history.
- Use a **sliding window** memory to control token cost; switch to Redis/Mongo/JDBC for persistent, stateful memory.

COMING NEXT (THE REAL GAME)

These four sessions were the foundation. Upcoming Spring AI classes cover image generation, audio, controlling the number of output tokens and the output format (structured output), embeddings, vector stores (PostgreSQL/pgvector, Redis), RAG and advanced RAG, and MCP. At least three or four more Spring AI classes are planned.

TELUSKO Telusko | AI Engineering and Agentic AI with Java and Spring Boot. Notes for Session 04.